

Lab 05 — Building a Linked List

DSA27 Lab Manual · Week 5 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Fall 2026 · Week 5

Contents

Part 0 — Before you start	2
Part 1 — Nodes by hand	4
Part 2 — <code>push_front</code> : $O(1)$ at the head	5
Part 3 — <code>append</code> : $O(n)$ without a tail	6
Part 4 — Walking: <code>find</code> and <code>__getitem__</code>	7
Part 5 — <code>insert_at</code> : walk, then splice	9
Part 6 — Removing: <code>pop_front</code> and <code>remove</code>	10
Part 7 — reverse: three references	11
Part 8 — Measure it: the $O(n^2)$ trap	13
Part 9 — Challenge: the <code>_tail</code> reference (not graded)	15
Part 10 — Exercises at a glance	16
Part 11 — Take-home practice (not graded)	17
Part 12 — Bridge to Lecture 06: stacks	17
Summary	18
Answers to the checkpoints	19

How to use this lab. Before each part, read the Lecture 05 section it names. Then **draw before you code**: boxes for nodes, arrows for references, on paper, before and after the operation. Predict at each **Checkpoint** (answers at the end), then write the method, then run the tests the part names. Build the class one method at a time, in the order below. Almost every linked-list bug lives in one of four places: the empty list, a one-node list, the first node and the last node. Test those by hand.

Duration	One 2-hour lab session, plus about 4 hours at home
You will write	<code>dsa/linked_list.py</code> — the <code>LinkedList</code> methods <code>push_front</code> , <code>append</code> , <code>insert_at</code> , <code>pop_front</code> , <code>remove</code> , <code>find</code> , <code>reverse</code> and <code>__getitem__</code>
Graded by	<code>tests/test_linked_list.py</code> (24 tests)
Connects to	Lecture 05 — Linked Lists; and Lecture 06 — Stacks

What you will be able to do

1. build a chain of `Node` objects by hand at the REPL, and say what every reference points at;
 2. write the traversal loop without thinking, and know when to stop **on** the last node and when to stop **after** it;
 3. insert at the front, in the middle and at the end, changing references in an order that never loses the rest of the list;
 4. remove from the front and by value, including the head special case;
 5. reverse a list in place with three references, and trace it on paper;
 6. keep `_size` correct in every method, so that `len` stays $O(1)$;
 7. measure why for `i in range(len(ll))`: `ll[i]` is $O(n^2)$, and write the $O(n)$ loop instead;
 8. say what a tail reference buys and what it costs.
-

Part 0 — Before you start

0.1 Environment check

From the DSA27 folder, with the virtual environment active (the prompt starts with `(.venv)`):

```
git pull
pytest -m "not challenge" -q
```

The second line must end with `passed`. If it does not, fix that first — Lab 01, Part 0, and the TA guide's table of environment problems.

0.2 What this lab stands on

The linked list is the one structure of the term that does not stand on another `dsa/` structure: it stores its data in **node objects**, the second of the three things the storage rule allows (Lecture 02). So there is no prerequisite test file to pass first. What you do need from earlier weeks:

- **Lab 01** — a name refers to an object; `b = a` copies nothing. `node.next` is exactly that: one more name bound to one more object.
- **Lab 03** — classes, `self`, special methods, and generators. The given `__iter__` is a generator.

- **Lecture 04** — `DynamicArray`. This week's costs are the mirror image of last week's, and you will compare them.

0.3 Read the skeleton

Open `dsa/linked_list.py`. Read every docstring: each one is part of the contract, and states the target cost as well as the behaviour.

Given — do not change	Yours
<code>Node(value, next=None)</code> , with <code>__slots__</code> and <code>__repr__</code>	<code>push_front(value)</code>
<code>LinkedList.__init__(values=())</code> — note that it calls <code>self.append</code>	<code>append(value)</code>
<code>__len__</code> — returns <code>self._size</code>	<code>insert_at(index, value)</code>
<code>__iter__</code> — the traversal loop, as a generator	<code>pop_front()</code>
<code>__repr__</code> — <code>LinkedList([1, 2, 3])</code>	<code>remove(value)</code>
	<code>find(value)</code>
	<code>reverse()</code>
	<code>__getitem__(index)</code>

Two things follow from the given code.

- **`__init__` calls `append`.** Until `append` works, `LinkedList([1, 2, 3])` raises `NotImplementedError`. Most tests build their list that way, so `append` comes early in the build order.
- **`__len__` returns `_size` and never counts.** So every method you write that adds or removes a node must update `self._size`. Nothing else will.

The object holds exactly two attributes: `self.head` (the first `Node`, or `None` when the list is empty) and `self._size`. The tests check `ll.head` directly, so there is **no sentinel node** in this exercise (Lecture 05, “Doubly linked, and sentinels”): `head` is the first *real* node.

The storage rule, here. Your methods may create `Node` objects and change references. They may not keep a Python `list`, `dict`, `set` or `deque` inside the `LinkedList`, and `reverse` in particular may not copy the values out into a list. A list as a *temporary* in your own REPL experiments is fine.

Run the tests once now, to see the starting point:

```
pytest tests/test_linked_list.py -q
```

Expect 22 failed, 2 passed. The two that pass test only the given code: `test_node_holds_a_value_and_a` and `test_empty_list`.

Part 1 — Nodes by hand

Read Lecture 05, “A node: a value and a link” and “Build one by hand”.

A linked list is nothing more than a chain of nodes, each holding a value and a reference to the next, with `None` after the last. Before you write a single method, build one yourself at the REPL. Everything in this lab is only a question of **which reference you change, and in what order**.

```
>>> from dsa.linked_list import Node
>>> c = Node(1)
>>> b = Node(7, c)
>>> a = Node(3, b)
>>> a.value, a.next.value, a.next.next.value
(3, 7, 1)
>>> a.next.next.next is None
True
>>> a.next is b
True
```

Notice the order you had to build it in: from the **end** backwards, because a node’s next must already exist when you create it.

Node has `__slots__ = ("value", "next")`. As well as saving memory (Lecture 05, “Memory: a node is not free”), this catches typing mistakes:

```
>>> a.nxt = Node(9)
AttributeError: 'Node' object has no attribute 'nxt' and no __dict__ for setting new attrib
```

Without `__slots__`, that line would silently create a new attribute, and your list would never change.

Draw it

On paper, draw the three nodes above as boxes with arrows, and label each arrow with the expression that reaches it (`a.next`, `a.next.next`, ...). Then let the course’s drawing helper draw the same chain. It takes plain values, head first:

```
from viz.draw import draw_linked_list

draw_linked_list([3, 7, 1], highlight=1, title="a.next is the node holding 7")
```

In the notebook `notebooks/05-linked-list.ipynb` the figure appears under the cell. In a script, add `import matplotlib.pyplot as plt` and `plt.show()` at the end. `highlight=i` colours the node at position `i` — use it to show where a pointer is while you walk.

Checkpoint 1. Predict each line:

```

c = Node("c"); b = Node("b", c); a = Node("a", b)
print(repr(a.next.next))
a.next = c
print((a.next.value, b.next.value))
d = Node("d", a)
print(d.next.next.value, d.next.next.next)

```

After `a.next = c`, what still refers to the node holding "b"?

Part 2 — push_front: O(1) at the head

Read Lecture 05, "At the front: O(1)".

A new first node needs two references changed, whatever the length of the list: the new node must point at the old first node, and head must point at the new node. Then `_size` goes up by one. Nothing is shifted, nothing is copied.

Draw it

Draw `head → 3 → 7 → None`. Add a box 5 above it. Draw the two arrows that change, numbered **1** and **2** in the order you will set them. Now try the other order on a second drawing, and follow the arrows from head.

Checkpoint 2. A student writes `push_front` with the two assignments the wrong way round:

```

def push_front(self, value):
    new = Node(value)
    self.head = new
    new.next = self.head
    self._size += 1

```

After `ll = LinkedList(); ll.push_front(3); ll.push_front(7)`, predict `len(ll)`, `ll.head.value` and `ll.head.next` is `ll.head`. What does `List(itertools.islice(ll, 5))` return — and what would plain `List(ll)` do?

Write it

1. Create the new node, linked to the **current** head.
2. Make head refer to the new node.
3. Add one to `_size`.

The rule from the lecture: **link the new node in before you let go of the old one**. Steps 1 and 2 also fit in a single assignment, because Python evaluates the whole right-hand side — including the old `self.head` — before it assigns: `Node(value, self.head)` already holds the old head when `self.head` changes.

The empty list needs no special case: when head is None, the new node's next is None, which is exactly right for a one-node list.

Test it

```
pytest tests/test_linked_list.py -v -k "node_holds or empty_list or push_front"
```

That selects 3 tests: `test_node_holds_a_value_and_a_link`, `test_empty_list` and `test_push_front_reverses_insertion_order`. (A plain `-k empty` would also pick up `test_pop_front_on_empty_raises`, which you have not written yet.)

When it fails

- **The test never finishes.** No dot, no F, nothing. You set head before linking the new node, so the new node points at itself, and the given `__iter__` walks round that one-node loop for ever — exactly Checkpoint 2 and question W5-S2. Press Ctrl+C; the traceback ends inside `__iter__`. Fix the order.
 - **`assert 0 == 3`**, with where `0 = len(LinkedList([3, 2, 1]))`. The values are right and the length is wrong: you forgot `self._size += 1`. The repr in the message is the list; the number is len.
-

Part 3 — append: $O(n)$ without a tail

Read Lecture 05, "At the end: $O(n)$ — unless you keep a tail".

To append, you must find the **last** node and point its next at the new node. The list holds no reference to the last node, so you walk from head — $O(n)$. This is the one walk in the lab that must stop **on** the last node rather than after it, so its loop condition is `while node.next is not None`.

The empty list has no last node at all. That is the special case: the new node simply becomes the head.

Draw it

Draw head $\rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \text{None}$. Put your finger on head and move it along one arrow at a time, stopping on the node whose next is None. Count your moves. Then do the same with `draw_linked_list([1, 2, 3], highlight=i)` for $i = 0, 1, 2$.

Checkpoint 3. With a correct `push_front` and `append`:

```
ll = LinkedList()
ll.push_front(2)
ll.append(3)
ll.push_front(1)
ll.append(4)
```

- (a) What is *LL*?
- (b) How many times does the walking step (`node = node.next`) run inside the last *append*?
- (c) `LinkedList(range(5))` calls *append* five times. How many walking steps in total? And for `LinkedList(range(1000))`?

Write it

1. Create the new node (its `next` is `None` — it will be last).
2. If the list is empty, make it the head.
3. Otherwise start at head and step along while the current node has a `next`; then link the current node — the last one — to the new node.
4. Add one to `_size`, in **both** branches.

Keep the docstring's challenge — a `_tail` reference — for later (Part 9). Get the $O(n)$ version passing first.

Test it

```
pytest tests/test_linked_list.py -v -k "append or constructor"
```

2 tests: `test_append_preserves_insertion_order` and `test_constructor_takes_an_iterable`. Once these pass, run the whole file: many tests that failed with `NotImplementedError` at `LinkedList(...)` now get as far as the method they are really testing.

When it fails

- **AttributeError: 'NoneType' object has no attribute 'next'** — in 19 of the 24 tests, because the constructor uses `append`. You walked without handling the empty list: `self.head` is `None`, and `None.next` does not exist. Add the empty case first.
- **AttributeError: 'NoneType' object has no attribute 'next' and no `__dict__` for setting new attributes** — note the second half: this is an *assignment* to `None.next`. You walked with `while node is not None`, which runs one step too far — off the end — so `node` is `None` when you try to link it. Stop **on** the last node: `while node.next is not None`.
- **Both tests pass, but later parts fail on lengths** — `assert 1 == 3` with where `1 = len(LinkedList([1, 2, 3]))`, or an `IndexError: 2 from 11[2]` on a three-value list. You added `_size += 1` in only one branch (here, only the empty one). The two `append` tests never check `len`; the later ones do.

Part 4 — Walking: find and `__getitem__`

Read Lecture 05, “The traversal loop” and “Indexing is $O(n)$ — that is the lesson”.

Every linked-list algorithm is the traversal loop plus a little bookkeeping:

```
node = self.head
while node is not None:
    # ... use node.value ...
    node = node.next
```

The given `__iter__` is this loop with `yield node.value` in the middle. `find` is this loop with an index counter and an early return. `__getitem__` is this loop stopped after `index` steps. Both walk the whole list in the worst case: $O(n)$. Here the loop stops **after** the last node (`node is not None`), because every node must be checked.

Draw it

`draw_linked_list(list(ll), highlight=i)` for $i = 0, 1, 2, \dots$ is exactly what `__getitem__` does in its head: to reach position 3 it must visit 0, 1 and 2 first. There is no arithmetic that jumps ahead — compare `DynamicArray[i]`, which is one address calculation.

Checkpoint 4. With `ll = LinkedList(["a", "b", "c", "b"])`, predict `ll.find("b")`, `ll.find("z")`, `ll[3]` and `ll[4]`. How many `node = node.next` steps does `ll[3]` take?

Write it

find(value) — walk with a node reference and an index that starts at 0. At each node, if its value equals `value`, return the index; otherwise step both on. If the loop ends, return -1. The **first** match wins.

__getitem__(index) —

1. If `index` is out of range (not $0 \leq \text{index} < \text{self._size}$), raise `IndexError`. `_size` makes this check $O(1)$: you know before walking.
2. Start at head and take exactly `index` steps.
3. Return that node's value.

The docstring, the tests and the lecture ask only for $0 \leq \text{index} < \text{len}(\text{self})$. Supporting negative indices as well (`ll[-1]`, by adding `self._size` first) is a reasonable extra, and it is what the worked solution does.

Test it

```
pytest tests/test_linked_list.py -v -k "find or getitem"
```

3 tests: `test_find`, `test_getitem` and `test_getitem_out_of_range_raises`.

When it fails

- `assert -1 == 2`, with where `-1 = find('c')` on `LinkedList(['a', 'b', 'c'])`. Your `find` loop is `while node.next is not None`: it stops **on** the last node without checking it, so the last value is never found (question W5-B1). Use `while node is not None`.

- **AttributeError: 'NoneType' object has no attribute 'next'** in `test_getitem_out_of_range`. No range check: walking five steps along a one-node list runs off the end. The test expects `IndexError` — check first, walk second.
-

Part 5 — `insert_at`: walk, then splice

Read Lecture 05, “In the middle: walk, then splice”.

A singly linked list can only be changed **from the node before** the change, because only that node holds the reference that must be redirected. To put a new node at index `i`, stop at index `i - 1` — call it `prev` — and splice in two assignments, in the safe order: the new node takes `prev`’s link first, then `prev` points at the new node.

Index 0 has no `prev`. It is `push_front` — so call `push_front`.

Draw it

Draw `head → 10 → 20 → 30 → None` and insert 15 at index 1. Mark `prev`. Number the two arrows that change. Then insert 40 at index 3 — the end — and check that the same two steps work when `prev.next` is `None`.

Checkpoint 5. *Predict what is printed, and what the last line does:*

```
ll = LinkedList([10, 20, 30])
ll.insert_at(3, 40)
ll.insert_at(1, 15)
ll.insert_at(0, 5)
print(ll, len(ll))
ll.insert_at(7, 99)
```

Write it

1. Validate: unless `0 ≤ index ≤ self._size`, raise `IndexError`. Note the `≤` at the top end: `index == len(self)` is legal and means “at the end”.
2. If `index == 0`, call `push_front(value)` and return. (It already updates `_size` — do not add one twice.)
3. Otherwise start `prev` at `head` and step `index - 1` times.
4. Splice the new node in after `prev`, taking `prev`’s old `next` as its own.
5. Add one to `_size`.

Test it

```
pytest tests/test_linked_list.py -v -k insert_at
```

5 tests: `test_insert_at` with index 0, 1 and 3, and `test_insert_at_rejects_out_of_range` with -1 and 5.

When it fails

- `assert [1, 2, 'x', 3] == [1, 'x', 2, 3]` for index 1, and `AttributeError: 'NoneType' object has no attribute 'next'` for index 3. You walked index steps instead of index - 1: you stopped **on** the node that should come after the new one, so the new node lands one place too far right — and at the end there is no node to stop on. Walk to the node **before** the gap.
- **Failed: DID NOT RAISE IndexError** for index -1, and an `AttributeError` for index 5. No validation. `range(-2)` is empty, so -1 quietly inserts after the head; 5 walks off the end. Check the range first, with `<=`.
- `assert 3 == 4`, with where `3 = len(LinkedList(['x', 1, 2, 3]))`, for index 0 only. Your `push_front` never updated `_size`; `insert_at(0, ...)` inherited the bug. Fix it there, not here.

Part 6 — Removing: `pop_front` and `remove`

Read Lecture 05, “From the front: $O(1)$ ” and “By value: skip over it”.

`pop_front` moves head one node on and returns the old head’s value. The old first node is no longer referenced by anything, so Python’s garbage collector reclaims it — you never free a node yourself. An empty list has no head to pop: the contract says `IndexError`.

`remove(value)` finds the node **before** the one holding `value` and bypasses it: `prev.next = prev.next.next`. The **head** is the special case — it has no node before it. (A sentinel node would remove the special case; this exercise does not use one, so you write it.)

Draw it

Draw `head → 3 → 7 → 1 → 9 → None` and `remove 7` (question W5-S3). Mark `prev` on the node holding 3 and cross out the arrow that changes. Then `remove 3` from the original list: where is `prev` now? There is none — which is exactly why the head needs its own branch.

Checkpoint 6. Predict the four values printed on the first line, then the second line, then what the last line does:

```
ll = LinkedList([4, 8, 4, 2])
print(ll.remove(4), ll.remove(5), ll.pop_front(), ll.remove(2))
print(ll, len(ll))
ll.pop_front()
ll.pop_front()
```

(The arguments of `print` are evaluated left to right.)

Write it

`pop_front()`

1. If the list is empty, raise `IndexError` (the solution's message is "pop from empty list").
2. Remember the head's value.
3. Move head to the second node (which may be `None`).
4. Subtract one from `_size`; return the remembered value.

remove(value)

1. If the list is empty, return `False`.
2. If the head holds value, move head on, subtract one from `_size`, return `True`.
3. Otherwise start prev at the head. While `prev.next` exists: if `prev.next.value == value`, bypass it, subtract one from `_size`, return `True`; else step prev on.
4. Nothing matched: return `False`.

Notice that the loop looks one node **ahead** (`prev.next.value`) — that is how it stays on the node before the match. Return exactly `True` or `False`: the tests use `is True`.

Test it

```
pytest tests/test_linked_list.py -v -k "pop_front or remove"
```

6 tests: `test_pop_front`, `test_pop_front_on_empty_raises`, `test_remove_first_match_only`, `test_remove_updates_the_length`, `test_remove_head` and `test_remove_absent_value`.

When it fails

- **AttributeError: 'NoneType' object has no attribute 'value'** in `test_pop_front_on_empty_`
You read `self.head.value` before checking for an empty list. The contract promises `IndexError`, not a crash.
- **assert False is True**, with where `False = remove(1)` on `LinkedList([1, 2, 3])`, in `test_remove_head` — and, in `test_remove_updates_the_length`, `assert 2 == 1` with where `2 = len(LinkedList([1, 3]))`. Your loop only ever looks at `prev.next`, so it never checks the head itself (question W5-B2): the second `remove(1)` found nothing. Add the head branch before the loop.
- **assert 3 == 1**, with where `3 = len(LinkedList([3]))`, in `test_remove_updates_the_length` only. The values are right and the length is not: you forgot `_size -= 1` in both branches, the head one and the loop one. Forget it in only one of them and the message is `assert 2 == 1` (still `LinkedList([3])`): the test removes once through each branch, so it catches either.

Part 7 — reverse: three references

Read Lecture 05, "Three references".

Reversal re-links the **same** nodes: every arrow is turned round, and the old last node becomes the head. You keep two references — `prev`, the head of the part already reversed, and `node`, the

head of the part still to do — plus a third, temporary one to remember what comes after node. One step moves one node across the line:

1. remember `node.next` — you are about to overwrite it;
2. point `node.next` back at `prev`;
3. advance: `prev` becomes `node`, `node` becomes the remembered next.

Start with `prev = None` (nothing reversed yet) and `node = self.head`. Stop when `node` is `None`. Then `prev` is the new head. $O(n)$ time, $O(1)$ extra space.

Draw it — on paper, before any code

This is homework item 4 of the lecture; do it now. Draw `1 → 2 → 3 → 4 → None` four times, one row per step, and fill in this table:

After step	prev	node	Reversed part	Part still to do
start	None	1	(empty)	1 → 2 → 3 → 4
1				
2				
3				
4				

In the middle of the reversal there are **two** chains, not one list, so draw each chain separately. To check your drawing at the REPL, a small helper that collects the values of a chain is enough (it is a REPL aid, not part of your class):

```
def chain(node):
    values = []
    while node is not None:
        values.append(node.value)
        node = node.next
    return values

draw_linked_list(chain(prev), title="reversed so far")
draw_linked_list(chain(node), title="still to do")
```

Checkpoint 7. (a) On `LinkedList([1, 2, 3, 4])`, after **two** steps of the loop, what are `prev.value`, `node.value`, `chain(prev)` and `chain(node)`?

(b) A student's loop is right but they forget the final `self.head = prev`. After `LL = LinkedList([1, 2, 3, 4]); LL.reverse()`, what do `print(LL)` and `len(LL)` show? Where are the other three nodes?

Write it

The three moves above, in a `while node is not None` loop, then set head. Step 1 is the one people forget: once `node.next = prev` has run, the only reference to the rest of the list is gone

— unless you saved it first. The empty list and the one-node list need no special case; check that by running your loop on paper for both.

Test it

```
pytest tests/test_linked_list.py -v -k "reverse and not push_front"
```

5 tests: `test_reverse`, `test_reverse_handles_short_lists` for `[]`, `[1]` and `[1, 2]`, and `test_reverse_relinks_rather_than_rebuilding`. A plain `-k reverse` selects 6, because `test_push_front_reverses_insertion_order` contains the word too.

When it fails

- **`assert [1] == [4, 3, 2, 1]`** (and `assert [1] == [2, 1]`). Either you forgot `self.head = prev` — head is still the old first node, which now points at None — or you stepped with `node = node.next` **after** turning the link round, so the step went back to `prev` (question W5-B4). Save the next node first; set head last.
- **`test_reverse_relinks_rather_than_rebuilding` fails** with two sets of ids that differ (Extra items in the left set: ...), while the other reverse tests pass. You built a new list of new nodes — for example with `push_front` for every value. The result looks right, but it is $O(n)$ extra space and not what was asked. Re-link the nodes you have.
- **The same test fails with the old last node must become the head: relink, do not copy values**, then `assert Node(3) is Node(1)` and where `Node(3) = LinkedList([3, 2, 1]).head`. You copied the values into a Python list and wrote them back into the same nodes in reverse order. The values look reversed, but the head is still the old **first** node object — it now holds 3, which is why its repr says `Node(3)`; the `Node(1)` on the right is the old last node, which still holds 1. That version uses $O(n)$ extra space and a Python list, against the docstring and the storage rule, and turns no link round at all. Re-link.

When all five pass, run the whole file:

```
pytest tests/test_linked_list.py -v
```

24 passed is the goal. The worked solution is in `solutions/dsa/linked_list.py` — open it **after** you have passed, or after an honest hour stuck, and compare it line by line with yours.

Part 8 — Measure it: the $O(n^2)$ trap

Read Lecture 05, “Indexing is $O(n)$ — that is the lesson” and “Measured, not asserted”.

`ll[i]` looks exactly like array indexing, and Python lets you write it because you wrote `__getitem__`. But every `ll[i]` starts again at head. So this innocent loop

```
for i in range(len(ll)):
    total += ll[i]
```

does $0 + 1 + 2 + \dots + (n - 1)$ steps: the triangular sum, $O(n^2)$. The loop that uses `__iter__` walks the list once: $O(n)$.

Checkpoint 8. For a list of $n = 1000$ nodes, how many `node = node.next` steps does the indexing loop above take in total? If n doubles, by roughly what factor should its time grow — and the time of `for value in LL`?

Put this in a script (or a notebook cell) and run it once all your tests pass. It builds each test list with `push_front` backwards, which is $O(n)$; building it with `LinkedList(range(n))` would itself be $O(n^2)$ — see the take-home question W5-K1.

```
from dsa.linked_list import LinkedList
from viz.complexity import measure, plot_growth

def build(n):
    ll = LinkedList()
    for value in reversed(range(n)):
        ll.push_front(value)
    return ll

def sum_by_index(ll):
    total = 0
    for i in range(len(ll)):
        total += ll[i]          # each ll[i] walks from the head
    return total

def sum_by_iteration(ll):
    total = 0
    for value in ll:           # one walk in total
        total += value
    return total

sizes = [500, 1000, 2000, 4000]
indexed = measure(sum_by_index, sizes, build)
iterated = measure(sum_by_iteration, sizes, build)
plot_growth({"for i in range(len(ll)): ll[i]": indexed,
            "for value in ll": iterated},
            reference=["n", "n^2"], loglog=True)
```

(In a script, finish with `import matplotlib.pyplot as plt; plt.show()`.)

What you should see. On the log-log axes both lines are straight, and the indexing line is about **twice as steep**: its slope matches the dashed $O(n^2)$ reference, the iteration line's matches $O(n)$.

Print the timings: each time n doubles, the iteration time roughly doubles and the indexing time roughly quadruples (often a little more, as the list outgrows the processor's cache). At $n = 4000$ the indexing loop is hundreds to thousands of times slower. Your exact numbers will differ; the **shapes** will not.

A second measurement in the same style: replace the two functions with building a list of n values by `append (LinkedList(range(n)))` and by `push_front (build(n))`, with `make_input=lambda n: n`. Which one bends?

The lesson for your own code, every week from now on: an interface does not tell you the cost. **Iterate a linked list; never index it in a loop.**

Part 9 — Challenge: the `_tail` reference (not graded)

Read Lecture 05, "At the end: $O(n)$ — unless you keep a tail" and "The costs, side by side".

The `append` docstring sets a challenge: keep a second reference, `_tail`, to the last node, and `append` becomes $O(1)$ — link after `_tail`, then move `_tail`. The price is that **every** method that can change the last node must keep `_tail` correct. Do this in a copy, or on a branch, once all 24 tests pass — the tests do not require it, and they must still pass afterwards.

Audit every method. For each, ask: *can this change which node is last, or make the list empty, or make it non-empty?*

Method	What <code>_tail</code> needs	Cost after the change
<code>__init__</code>	start as <code>None</code> — before the loop that appends	—
<code>append</code>	empty list: head and tail are the new node; else link after <code>_tail</code> , move <code>_tail</code>	$O(1)$ (was $O(n)$)
<code>push_front</code>	on an empty list the new node is also the tail	$O(1)$
<code>insert_at</code>	at <code>index == len(self)</code> the new node is the new tail — or just call <code>append</code>	$O(n)$
<code>pop_front</code>	popping the only node must clear <code>_tail</code>	$O(1)$
<code>remove</code>	removing the last node: <code>_tail</code> moves back to <code>prev</code> — which you hold only because you walked to it	$O(n)$
<code>reverse</code>	the old head becomes the tail	$O(n)$

The row that matters is `remove`. Removing the last node is still $O(n)$ in a singly linked list, tail or no tail: the new last node is the one *before* it, and only a walk from the head finds it (question

W5-M16). A **doubly** linked list fixes that, for the price of a second reference in every node.

Checkpoint 9. A student adds `_tail` and updates `__init__`, `append`, `push_front` and `pop_front`, but forgets `remove`. Predict:

```
ll = LinkedList([1, 2, 3])
print(ll.remove(3))
ll.append(4)
print(list(ll), len(ll))
```

That is the general lesson of the lecture: every extra reference buys speed somewhere and costs care everywhere. In Week 7 you will need exactly this structure — head for one end, tail for the other — for a linked queue.

Part 10 — Exercises at a glance

Method	Target cost	The trap	Tests (-k)
<code>push_front(value)</code>	$O(1)$	link the new node before moving head; wrong order is a self-loop	<code>push_front</code> (1)
<code>append(value)</code>	$O(n)$; $O(1)$ with <code>_tail</code>	the empty list has no last node; stop on the last node	<code>append</code> or constructor (2)
<code>find(value)</code>	$O(n)$	stop after the last node, or the last value is never found; -1 when absent	<code>find</code> (1)
<code>__getitem__(index)</code>	$O(n)$	<code>IndexError</code> before walking; never index in a loop	<code>getitem</code> (2)
<code>insert_at(index, value)</code>	$O(n)$	index 0 is <code>push_front</code> ; stop at <code>index - 1</code> ; <code>index == len</code> is legal	<code>insert_at</code> (5)
<code>pop_front()</code>	$O(1)$	<code>IndexError</code> when empty; <code>_size -= 1</code>	<code>pop_front</code> (2)
<code>remove(value)</code>	$O(n)$	the head has no prev; <code>_size -= 1</code> in both branches; return True/False	<code>remove</code> (4)

Method	Target cost	The trap	Tests (-k)
<code>reverse()</code>	$O(n)$ time, $O(1)$ space	save next before overwriting it; set head last; re-link, do not rebuild	<code>reverse</code> and <code>not push_front</code> (5)

Every method that adds or removes a node: **update_size**.

Part 11 — Take-home practice (not graded)

The Week 5 question bank is `docs/question-bank/week05-questions.md`, with answers in `week05-answers.md` — try first, then check.

Coding problems W5-C1 to W5-C5 are in `practice/week05.py`. Each takes the first Node of a chain and must walk and re-link, never copy into a Python list:

```
pytest tests/test_practice_week05.py -v
```

- **W5-C1** `middle_value` and **W5-C5** `kth_from_end` — two references moving at different speeds, or a fixed distance apart.
- **W5-C2** `has_cycle` — detect exactly the self-loop of Checkpoint 2, in $O(1)$ extra space.
- **W5-C3** `merge_sorted_chains` and **W5-C4** `remove_duplicates_sorted` — re-linking, the reverse skill again.

Trace and state questions to do on paper, with the drawings:

- **W5-S1** — a sequence of every operation you wrote; give the list after each line, `len` and a `find`.
- **W5-S2** — the wrong-order `push_front`, on a list that was not empty.
- **W5-T4** — `reverse()` on $1 \rightarrow 2 \rightarrow 3$, step by step.
- **W5-K1** — why building a list with `append` in a loop is $\Theta(n^2)$, and two ways to make it $\Theta(n)$. (You measured one of them in Part 8.)

Worked solutions: `solutions/practice/week05.py`, and `pytest --solutions tests/test_practice_w` runs the tests on them. As always, they are for **after** you have tried.

Part 12 — Bridge to Lecture 06: stacks

Lecture 06 is the **stack**: last in, first out, with `push`, `pop` and `peek`, all $O(1)$. Every stack operation touches only one end, the **top**. So the question is: which end of which structure is $O(1)$?

You have just built the answer twice over. On your `DynamicArray` (Week 4) the cheap end is the **end**. On your `LinkedList` the cheap end is the **head**: `push_front` and `pop_front` are $O(1)$, worst case, with no resizing. A stack is a linked list with the top at the head.

Try it with the class you have just finished:

```
>>> from dsa.linked_list import LinkedList
>>> stack = LinkedList()
>>> for token in ["(", "[", "{"]:
...     stack.push_front(token)           # push
...
>>> stack
LinkedList(['{', '[', '('])
>>> stack.head.value                       # peek: the last one pushed
'{'
>>> stack.pop_front(), stack.pop_front() # pop, pop
('{', '[')
>>> len(stack)
1
```

The last bracket opened is the first one out — which is exactly what checking "(a[b]{c})" needs, and the first application in Lecture 06.

Before the lecture, think about the other choice: put the top at the **tail** of your linked list instead. Which of push and pop stays $O(1)$ — even with a `_tail` — and which becomes $O(n)$? (Part 9 has the answer.)

`dsa/stack.py` itself is built on your `DynamicArray`, with the top at the end; Lecture 06 compares the two honest designs, and one dishonest one, by measurement.

Summary

Idea	The one line to keep
Structure	Nodes (a value and a next), a head, and <code>_size</code> . No sentinel here.
Traversal	<code>while node is not None: ... node = node.next</code> — stop after the last node to visit all; <code>while node.next is not None</code> stops on it, for append.
<code>push_front</code>	$O(1)$. Link the new node to the old head before moving head.
<code>append</code>	$O(n)$ — walk to the last node; the empty list is the special case. $O(1)$ with <code>_tail</code> .
<code>insert_at</code>	$O(n)$. Index 0 is <code>push_front</code> ; otherwise stop at <code>index - 1</code> and splice.
<code>pop_front</code>	$O(1)$. <code>IndexError</code> when empty.
<code>remove</code>	$O(n)$. Change it from the node before ; the head is the special case.

Idea	The one line to keep
reverse	Three references; save next first; head = prev last. $O(n)$ time, $O(1)$ space.
__getitem__	$O(n)$ by contract. Indexing in a loop is $O(n^2)$: iterate instead.
_size	Every method that adds or removes a node updates it — len depends on nothing else.
_tail	Buys $O(1)$ append; costs care in every method; removing the last node stays $O(n)$.

Answers to the checkpoints

Checkpoint 1.

```
Node('c')
('c', 'c')
c None
```

a.next.next is the node holding "c", and Node.__repr__ shows only its value. After a.next = c, both a and b point at c. Nothing in the chain refers to the node holding "b" any more — only the REPL name b keeps it alive; in a list, it would be gone. $d \rightarrow a \rightarrow c \rightarrow \text{None}$, so d.next.next is c, whose next is None.

Checkpoint 2. len(ll) is 2, ll.head.value is 7, and ll.head.next is ll.head is True. list(itertools.islice(ll, 5)) is [7, 7, 7, 7, 7]: the head points at itself, so __iter__ yields 7 for ever, and islice stops it after five. Plain list(ll) never finishes — it grows a Python list until you press Ctrl+C (or memory runs out). The node holding 3 is unreachable. With the correct order, the list is LinkedList([7, 3]).

Checkpoint 3. (a) LinkedList([1, 2, 3, 4]). (b) **2**: starting on node 1, the walk steps to 2, then to 3, whose next is None. (c) **6**: the five appends walk 0, 0, 1, 2 and 3 steps (the first finds an empty list, the second starts on the only node). For 1000 values, $0 + 0 + 1 + \dots + 998 = \mathbf{498,501}$ steps — building by append in a loop is $O(n^2)$.

Checkpoint 4. ll.find("b") is 1 — the **first** match; ll.find("z") is -1; ll[3] is 'b'; ll[4] raises IndexError (the solution's message is IndexError: 4), because the valid indices are 0 to 3. ll[3] takes **3** steps: from the node at 0 to 1, 2 and 3.

Checkpoint 5. It prints LinkedList([5, 10, 15, 20, 30, 40]) 6. insert_at(3, 40) on a 3-node list is legal — it appends. insert_at(7, 99) raises IndexError (the solution's message is IndexError: 7): the valid indices are 0 to len(ll), which is 6.

Checkpoint 6. The first line is True False 8 True: remove(4) removes only the **first** 4, the head ([8, 4, 2]); 5 is absent; pop_front() returns 8 ([4, 2]); remove(2) removes the last node. The second line is LinkedList([4]) 1. The first pop_front() then empties the list, and the second raises IndexError: pop from empty list.

Checkpoint 7. (a) `prev.value` is 2, `node.value` is 3, `chain(prev)` is `[2, 1]` and `chain(node)` is `[3, 4]`: two nodes have crossed the line, and node 1 now points at `None`. (b) `print(l1)` shows `LinkedList([1])` while `len(l1)` is still 4: head is still the old first node, which the reversal pointed at `None`. The other three nodes are correctly reversed, `4 → 3 → 2 → 1`, but nothing refers to node 4 — only `prev` did, and it vanished when the method returned — so they are unreachable.

Checkpoint 8. $0 + 1 + \dots + 999 = 999 \times 1000 / 2 = \mathbf{499,500}$ steps, against 1000 for `for value in l1`. Doubling `n` roughly **quadruples** the indexing loop's time ($O(n^2)$) and roughly **doubles** the iteration's ($O(n)$).

Checkpoint 9. It prints `True`, then `[1, 2]` 3. `remove(3)` unlinked the last node but left `_tail` pointing at it. `append(4)` then linked the new node after that **detached** node, and increased `_size` — so the list the head reaches is `[1, 2]`, while `len` says 3. No exception, just a silently wrong list: the most dangerous kind of bug.